

Evidencias de pruebas

Evidencia 1 Ordenamiento y búsqueda por rango en una bitácora de eventos

Pamela Hernández Camacho A01771362

20 de septiembre de 2026

1. Plan de pruebas

El objetivo de estas pruebas fue comprobar que el programa leyera la bitácora completa (6,818 registros), ordenara correctamente con los siete algoritmos y que la búsqueda por rango devolviera exactamente los registros correctos, incluso en los casos límite. Se probó lo siguiente:

Qué se probó	Por qué importa	Captura
Los 14 casos: 7 algoritmos $\times$ 2 archivos	Verificar que todos ordenan bien y comparar sus tiempos con mi predicción	1
Búsqueda por rango normal	Comprobar que devuelve los registros esperados y guarda range607.txt	2
Rango vacío y bordes de fechas	Los casos límite (primer y último registro, fuera del archivo) son donde más se equivocan las búsquedas	3
Fechas con formato inválido	El programa debe avisar y volver a preguntar, sin caerse	4
Opciones de menú inválidas	Igual: no debe romperse con entradas erróneas	5
Timestamps duplicados	No perder ni duplicar registros que tienen exactamente la misma fecha y hora	6

Entorno: macOS, compilador g++ (Apple clang), C++17 con optimización -O2. Los archivos tienen 6,818 registros cada uno.

2. Casos de prueba ejecutados

2.1 Ordenar y buscar un rango normal

Qué se ejecutó: ordenar log607-2.txt con Inserción y buscar del 16 de marzo de 2025 00:00:00 al 16 de marzo de 2025 23:59:59.

Resultado esperado: 6 registros, todos del 16 de marzo de 2025; output607.txt empieza con el registro más antiguo (Sep 08 2024) y range607.txt contiene esos 6 registros.

```

● pamelahc@MacBook-Pro-de-Pam-2 Evidence1 % clear
● pamelahc@MacBook-Pro-de-Pam-2 Evidence1 % printf '1\n2\n4\n1\nprueba\n2\nMar 16 2025 00:00:00\nMa
r 16 2025 23:59:59\n0\n' | ./build/bitacora | tail -22; echo "---- output607.txt (primeras 3 linea
s):"; head -3 out/output607.txt; echo "---- range607.txt:"; cat out/range607.txt
=> Tu predicción COINCIDIÓ
Datos ordenados guardados en out/output607.txt

===== MENÚ =====
1) Ordenar un archivo
2) Buscar por rango de fechas
0) Salir
Opción: Formato de fecha: Sep 29 2024 14:37:38
Fecha/hora de inicio: Fecha/hora de fin: Registros en el rango: 6
Mar 16 2025 03:38:51 10.14.128.93 Failed Login Attempts
Mar 16 2025 04:18:43 10.14.234.100 Human error
Mar 16 2025 12:21:58 10.14.176.121 Port Scanning Activity
Mar 16 2025 16:58:11 10.14.187.166 Configuration errors
Mar 16 2025 16:58:11 10.14.181.9 Anomalous User Activity
Mar 16 2025 18:36:02 10.14.183.10 Cross-Site Scripting
Resultado guardado en out/range607.txt

===== MENÚ =====
1) Ordenar un archivo
2) Buscar por rango de fechas
0) Salir
Opción: Hasta luego.
--- output607.txt (primeras 3 lineas):
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Sep 08 2024 04:37:37 10.14.248.79 Configuration errors
Sep 08 2024 04:44:15 10.14.135.12 Application error
--- range607.txt:
Mar 16 2025 03:38:51 10.14.128.93 Failed Login Attempts
Mar 16 2025 04:18:43 10.14.234.100 Human error
Mar 16 2025 12:21:58 10.14.176.121 Port Scanning Activity
Mar 16 2025 16:58:11 10.14.187.166 Configuration errors
Mar 16 2025 16:58:11 10.14.181.9 Anomalous User Activity
Mar 16 2025 18:36:02 10.14.183.10 Cross-Site Scripting
○ pamelahc@MacBook-Pro-de-Pam-2 Evidence1 % █

```

Resultado obtenido: Coincidió con lo esperado. La búsqueda devolvió 6 registros, todos del 16 de marzo de 2025, y los guardó en range607.txt. El archivo output607.txt empieza con el registro más antiguo (Sep 08 2024)

2.2 Rango vacío y bordes de fechas

Qué se ejecutó: siete búsquedas seguidas sobre los datos ordenados. Los resultados aparecen en el mismo orden que esta tabla:

Número	Rango consultado	Resultado esperado
1	Del primer registro (Sep 08 2024 00:22:43) al último (Sep 07 2026 23:50:40)	6818 registros (todos)
2	Solo el primer registro (inicio = fin = Sep 08 2024 00:22:43)	1 registro
3	Solo el último registro (inicio = fin = Sep 07 2026 23:50:40)	1 registro
4	Un segundo antes del primer registro (Sep 08 2024 00:22:42)	0 (rango vacío)

Número	Rango consultado	Resultado esperado
5	Después del último registro (Sep 07 2026 23:50:41 a Sep 08 2026 00:00:00)	0 (rango vacío)
6	Hueco real entre dos registros (Jul 10 2025 12:00:00 a Jul 11 2025 09:00:00)	0 (rango vacío)
7	Inicio posterior al fin (Sep 09 2024 a Sep 08 2024)	Mensaje: el inicio es posterior al fin

```
● pamelahe@MacBook-Pro-de-Pam-2 Evidence1 % printf '1\n2\n4\n1\nprueba\n2\nSep 08 2024 00:22:43\nSep 07 2026 23:50:40\n2\nSep 08 2024 00:22:43\nSep 08 2024 00:22:43\n2\nSep 07 2026 23:50:40\nSep 07 2026 23:50:40\n2\nSep 08 2024 00:22:42\nSep 08 2024 00:22:42\n2\nSep 07 2026 23:50:41\nSep 08 2026 00:00:00\n2\nJul 10 2025 12:00:00\nJul 11 2025 09:00:00\n2\nSep 09 2024 00:00:00\nSep 08 2024 00:00:00\n0\n' | ./build/bitacora | grep -E "Registros en el rango|Rango vacío|posterior"
Fecha/hora de inicio: Fecha/hora de fin: Registros en el rango: 6818
Fecha/hora de inicio: Fecha/hora de fin: Registros en el rango: 1
Fecha/hora de inicio: Fecha/hora de fin: Registros en el rango: 1
Fecha/hora de inicio: Fecha/hora de fin: Registros en el rango: 0
Rango vacío: no hay registros entre esas fechas.
Fecha/hora de inicio: Fecha/hora de fin: Registros en el rango: 0
Rango vacío: no hay registros entre esas fechas.
Fecha/hora de inicio: Fecha/hora de fin: Registros en el rango: 0
Rango vacío: no hay registros entre esas fechas.
Fecha/hora de inicio: Fecha/hora de fin: El inicio es posterior al fin.
○ pamelahe@MacBook-Pro-de-Pam-2 Evidence1 %
```

Resultado obtenido: Coincidió con lo esperado ya que el rango completo devolvió 6818 registros, el primer y el último registro se encontraron por separado (1 cada uno) y los tres rangos fuera del archivo o en un hueco devolvieron 0 con aviso de rango vacío. Cuando el inicio era posterior al fin, el programa avisó en lugar de buscar.

2.3 Formatos de fecha inválidos

¿Qué se ejecutó?: siete fechas de inicio distintas (con fin válido Dec 31 2026 00:00:00). El programa debe rechazar las inválidas con «Fecha inválida» y no caerse.

Fecha de inicio probada	Motivo	Resultado esperado
Feb 30 2025 10:00:00	Febrero no tiene 30 días	Fecha inválida
Feb 29 2025 10:00:00	2025 no es bisiesto	Fecha inválida
Feb 29 2024 10:00:00	2024 sí es bisiesto	Válida (6818)
Sep xx 2024 10:00:00	Letras donde van números	Fecha inválida
Sep 29 2024 25:00:00	Hora 25	Fecha inválida
31/12/2025 10:00:00	Otro formato de fecha	Fecha inválida
Sep 9 2024 10:00:00	Falta el cero de 09	Fecha inválida

```

pamela@MacBook-Pro-de-Pam-2 Evidence1 % printf '1\n2\n4\n1\nprueba\n2\nFeb 30 2025 10:00:00\nDec 31 2026 00:00:00\n2\nFeb 29 2025 10:00:00\nDec 31 2026 00:00:00\n2\nFeb 29 2024 10:00:00\nDec 31 2026 00:00:00\n2\nSep xx 2024 10:00:00\nDec 31 2026 00:00:00\n2\nSep 29 2024 25:00:00\nDec 31 2026 00:00:00\n2\n31/12/2025 10:00:00\nDec 31 2026 00:00:00\n2\nSep 9 2024 10:00:00\nDec 31 2026 00:00:00\n0\n' | ./build/bitacora | grep -E "Fecha inválida|Registros en el rango"
Fecha/hora de inicio: Fecha/hora de fin: Fecha inválida.
Fecha/hora de inicio: Fecha/hora de fin: Fecha inválida.
Fecha/hora de inicio: Fecha/hora de fin: Registros en el rango: 6818
Fecha/hora de inicio: Fecha/hora de fin: Fecha inválida.
Fecha/hora de inicio: Fecha/hora de fin: Fecha inválida.
Fecha/hora de inicio: Fecha/hora de fin: Fecha inválida.
Fecha/hora de inicio: Fecha/hora de fin: Fecha inválida.
pamela@MacBook-Pro-de-Pam-2 Evidence1 %

```

Resultado obtenido: Coincidió con lo esperado porque el programa rechazó seis fechas inválidas con "Fecha inválida" sin caerse, y aceptó Feb 29 2024 porque 2024 es bisiestro.

2.4 Opciones de menú inválidas

Qué se ejecutó: en el menú principal se escribió texto (abc), un número fuera de rango (9) y una línea vacía.

Resultado esperado: el programa responde «Opción inválida» y vuelve a preguntar, sin cerrarse.

```

pamela@MacBook-Pro-de-Pam-2 Evidence1 % printf 'abc\n9\n\n0\n' | ./build/bitacora | grep -E "Opción inválida|Hasta luego"
Opción: Opción inválida. Escribe un número entre 0 y 2.
Opción: Opción inválida. Escribe un número entre 0 y 2.
Opción: Opción inválida. Escribe un número entre 0 y 2.
Opción: Hasta luego.
pamela@MacBook-Pro-de-Pam-2 Evidence1 %

```

Resultado obtenido: Coincidió con lo esperado. Ante "abc", "9" y una línea vacía, respondió "Opción inválida" y volvió a preguntar.

2.5 Timestamps duplicados

El archivo log607-2.txt tiene registros con exactamente la misma fecha y hora. La política del programa es que el rango es cerrado: [inicio, fin]. Si hay registros que coinciden con el inicio o con el fin, todos entran. Además, se implementa con dos búsquedas binarias: limiteInferior devuelve la primera posición cuya fecha es mayor o igual a la buscada y limiteSuperior devuelve la primera posición cuya fecha es estrictamente mayor. Por último, el rango son las posiciones desde limiteInferior hasta limiteSuperior sin incluir esta última por lo que no se pierde ni se repite ningún registro.

Qué se ejecutó: ordenar log607-2.txt y buscar con inicio y fin iguales a Feb 13 2025 07:01:11 (una fecha que aparece dos veces).

Resultado esperado: 2 registros, ambos con la fecha Feb 13 2025 07:01:11, en el mismo orden en que venían en el archivo original.

```

pamela@MacBook-Pro-de-Pam-2 Evidence1 % printf '1\n2\n4\n1\nprueba\n2\nFeb 13 2025 07:01:11\nFeb 13 2025 07:01:11\n0\n' | ./build/bitacora | grep -E "Registros en el rango|^ Feb"
Fecha/hora de inicio: Fecha/hora de fin: Registros en el rango: 2
Feb 13 2025 07:01:11 10.14.17.247 Data Exfiltration Attempts
Feb 13 2025 07:01:11 10.14.121.243 Data corruption
pamela@MacBook-Pro-de-Pam-2 Evidence1 %

```

Resultado obtenido: Coincidió con lo esperado. Con inicio y fin en Feb 13 2025 07:01:11, el programa devolvió los 2 registros duplicados, sin perder ni repetir ninguno y en su orden original.

3. Resultados de tiempos

Se ejecutó cada uno de los siete algoritmos con cada uno de los dos archivos (14 corridas). Antes de cada corrida se registró una predicción (Rápida: menos de 5 ms, Media: de 5 a 50 ms, Lenta: más de 50 ms). En la tabla, archivo 1 es log607-1.txt (desordenado) y archivo 2 es log607-2.txt (casi ordenado).

```

pamela@MacBook-Pro-de-Pam-2 Evidence1 % grep -E "^Algoritmo|^Archivo|^Tiempo|^>=" out/corridas607.txt | past
e -d' ' - - - | sed 's/Algoritmo: //;s/Archivo: data\\//;s/Tiempo de ordenamiento: //;s/>= Tu predicción //'
Burbuja log607-1.txt 82.688041 ms COINCIDIÓ
Burbuja log607-2.txt 0.398625 ms COINCIDIÓ
Selección log607-1.txt 105.139250 ms COINCIDIÓ
Selección log607-2.txt 98.819750 ms COINCIDIÓ
Swap Sort log607-1.txt 85.513333 ms COINCIDIÓ
Swap Sort log607-2.txt 20.970042 ms NO coincidió
Inserción log607-1.txt 17.866167 ms NO coincidió
Inserción log607-2.txt 0.108375 ms COINCIDIÓ
Shell log607-1.txt 0.734708 ms COINCIDIÓ
Shell log607-2.txt 0.260292 ms COINCIDIÓ
Merge log607-1.txt 0.649666 ms COINCIDIÓ
Merge log607-2.txt 0.457417 ms COINCIDIÓ
Quick log607-1.txt 0.544209 ms COINCIDIÓ
Quick log607-2.txt 0.351459 ms COINCIDIÓ
pamela@MacBook-Pro-de-Pam-2 Evidence1 %

```

Algoritmo	Archivo	Predicción	Tiempo	Medido	¿Coincidió?
Burbuja	1	Lenta	82.7 ms	Lenta	sí
Burbuja	2	Rápida	0.40 ms	Rápida	sí
Selección	1	Lenta	105.1 ms	Lenta	sí
Selección	2	Lenta	98.8 ms	Lenta	sí
Swap Sort	1	Lenta	85.5 ms	Lenta	sí
Swap Sort	2	Lenta	21.0 ms	Media	NO
Inserción	1	Lenta	17.9 ms	Media	NO
Inserción	2	Rápida	0.11 ms	Rápida	sí
Shell	1	Rápida	0.73 ms	Rápida	sí
Shell	2	Rápida	0.26 ms	Rápida	sí
Merge	1	Rápida	0.65 ms	Rápida	sí
Merge	2	Rápida	0.46 ms	Rápida	sí
Quick	1	Rápida	0.54 ms	Rápida	sí
Quick	2	Rápida	0.35 ms	Rápida	sí

Coincidieron 12 de 14 predicciones. Las predicciones de esta tabla se hicieron con una regla teórica (si el algoritmo es $O(n^2)$ para ese archivo, Lenta; si es $O(n \log n)$ o está en su mejor caso,

Rápida). Cada tiempo es una sola corrida, por lo que las diferencias menores a 1 ms son ruido de medición.

4. Conclusión

- Burbuja (con bandera) e Inserción se benefician muchísimo del archivo casi ordenado: Burbuja pasó de 82.7 ms a 0.40 ms e Inserción de 17.9 ms a 0.11 ms. Su trabajo depende de cuántos pares de registros están en orden contrario: hay 17,547 en el archivo 2, contra 11,593,691 en el archivo 1.
- Selección casi no cambia (105.1 ms contra 98.8 ms), porque siempre revisa todos los registros que quedan a su derecha, sin importar el orden.
- Swap Sort mejora a la mitad o menos (85.5 ms a 21.0 ms): hace muchos menos intercambios (11,593,691 contra 17,547), pero las comparaciones siguen siendo las mismas.
- Shell, Merge y Quick son rápidos en ambos archivos (menos de 1 ms) y casi no cambian entre uno y otro.
- Quick Sort con el último registro como pivote tardó 41.2 ms con el archivo casi ordenado, contra 1.26 ms con el desordenado, porque el pivote quedaba casi siempre en un extremo y las particiones salían muy desiguales. Al escoger el pivote con la mediana de tres, bajó a 0.62 ms.
- Dos predicciones no coincidieron (Inserción con el archivo 1 y Swap Sort con el archivo 2): la complejidad indica cómo crece el tiempo, no cuánto tarda con un tamaño concreto. Con 6,818 registros, un algoritmo $O(n^2)$ todavía puede terminar en milisegundos.
- Recomendación: para bitácoras casi ordenadas conviene un algoritmo que aproveche el orden previo, como Inserción; para datos de orden desconocido, Merge o Quick con mediana de tres. Para consultar rangos, ordenar una vez y usar búsqueda binaria.